Django Complete Tutorial Notes — Part 1–8 + First Contribution + Practice Project

Level: Beginner to Advanced

Language: Simple English with Bangla explanations

Framework: Django

Table of Contents

1. Django Introduction
2. Part 1 — Project, App, Views and URLs
3. Part 2 — Database, Models, ORM and Admin
4. Part 3 — Views and Templates
5. Part 4 — Forms, Voting and Generic Views
6. Part 5 — Automated Testing
7. Part 6 — Static Files
8. Part 7 — Django Admin Customization
9. Part 8 — Third-party Packages and Debug Toolbar
10. First Contribution to Django
11. Important Commands and Practice Project

---

1. Django Introduction

What is Django?

Django is a high-level Python web framework used to build secure, maintainable, and scalable web applications.

Main Features

- Built-in authentication and authorization
- Object-Relational Mapper (ORM)
- Automatic administration interface
- URL routing
- Template engine
- Form validation
- Security protections
- Automated testing support

- Reusable application architecture

What is MVT?

Django follows the Model-View-Template (MVT) architectural pattern.

- **Model:** Defines data and database interactions.
- **View:** Processes requests and prepares responses.
- **Template:** Presents information to users.

Request Flow

Browser → URL Configuration → View → Model (if needed) → Template (if needed) → HTTP Response

বাংলা ব্যাখ্যা: Django ব্যবহার করে Python দিয়ে dynamic website, REST API এবং full-stack web application তৈরি করা যায়। MVT architecture application-এর code-কে আলাদা দায়িত্বে ভাগ করে।

Django vs Flask

Django	Flask
Full-featured framework	Lightweight framework
Built-in ORM and admin	Extensions commonly provide additional features
Structured application conventions	Flexible application structure
Suitable for many full-stack applications	Suitable for small services and customized

	architectures

---

2. Part 1 — Project, App, Views and URLs

Step 1: Create a Virtual Environment

```
python -m venv .venv
```

Activate on Windows:

```
.venv\Scripts\activate
```

Activate on Linux or macOS:

```
source .venv/bin/activate
```

Step 2: Install Django

```
python -m pip install django
```

Check the installed version:

```
python -m django --version
```

Step 3: Create a Project

```
django-admin startproject mysite
cd mysite
```

Step 4: Run the Development Server

```
python manage.py runserver
```

Open: http://127.0.0.1:8000/

Step 5: Create an Application

```
python manage.py startapp polls
```

A Django project can contain multiple applications.

Step 6: Project Structure

```
mysite/
├── manage.py
```

```
├── mysite/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
└── polls/
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    └── views.py
```

Step 7: Register the Application

In `mysite/settings.py`:

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "polls",
]
```

Step 8: Create a Function-Based View

File: `polls/views.py`

```
from django.http import HttpResponse


def home(request):
    return HttpResponse("Welcome to Django!")
```

Step 9: Configure Application URLs

Create `polls/urls.py`:

```
from django.urls import path
from . import views

app_name = "polls"
```

```
urlpatterns = [
    path("", views.home, name="home"),
]
```

Step 10: Connect Application URLs

File: `mysite/urls.py`

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("polls/", include("polls.urls")),
]
```

Visit: http://127.0.0.1:8000/polls/

Important Concepts

- `request` : Contains incoming HTTP request information.
- `HttpResponse` : Returns an HTTP response.
- `path()` : Maps a URL pattern to a view.
- `include()` : Includes another URL configuration.
- `app_name` : Defines an application namespace.
- `name` : Defines a URL pattern's name.

বাংলা ব্যাখ্যা: Project হলো সম্পূর্ণ Django website-এর configuration। App হলো নির্দিষ্ট feature-এর জন্য তৈরি একটি reusable component।

---

3. Part 2 — Database, Models, ORM and Admin

What is a Model?

A Django model is a Python class that defines the structure and behavior of stored data.

File: `polls/models.py`

```
from django.db import models


class Question(models.Model):
    question_text = models.CharField(max_length=200)
    pub_date = models.DateTimeField()
```

```
    def __str__(self):
        return self.question_text


class Choice(models.Model):
    question = models.ForeignKey(
        Question,
        on_delete=models.CASCADE,
        related_name="choices",
    )
    choice_text = models.CharField(max_length=200)
    votes = models.PositiveIntegerField(default=0)

    def __str__(self):
        return self.choice_text
```

Common Model Fields

Field	Purpose
`CharField`	Short text
`TextField`	Long text
`IntegerField`	Integer values
`PositiveIntegerField`	Non-negative integer-oriented field
`BooleanField`	True or false
`DateTimeField`	Date and time

`DateField`	Date
`EmailField`	Email-formatted text
`ForeignKey`	Many-to-one relationship
`OneToOneField`	One-to-one relationship
`ManyToManyField`	Many-to-many relationship

Model Relationships

One question can have multiple choices.

```
question = Question.objects.get(pk=1)
choices = question.choices.all()
```

The `related_name="choices"` setting defines the reverse relationship.

OneToOneField

Useful when one model should have at most one associated record.

```
from django.conf import settings
from django.db import models


class UserProfile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
```

```
    )
    bio = models.TextField(blank=True)
```

ManyToManyField

A course can have multiple students, and a student can enroll in multiple courses.

```
class Student(models.Model):
    name = models.CharField(max_length=100)


class Course(models.Model):
    title = models.CharField(max_length=100)
    students = models.ManyToManyField(
        Student,
        related_name="courses",
    )
```

Migrations

Create migration files:

```
python manage.py makemigrations
```

Apply migrations:

```
python manage.py migrate
```

Check migration status:

```
python manage.py showmigrations
```

Migrations keep the database schema synchronized with model changes.

Django ORM

The ORM allows database operations through Python objects.

Create

```
from django.utils import timezone
from polls.models import Question

Question.objects.create(
    question_text="What is Django?",
    pub_date=timezone.now(),
)
```

Retrieve All Objects

```
Question.objects.all()
```

Retrieve One Object

```
question = Question.objects.get(pk=1)
```

`get()` raises `Question.DoesNotExist` if no matching object exists.

Filter

```
Question.objects.filter(
    question_text__icontains="django"
)
```

Order By

```
Question.objects.order_by("-pub_date")
```

Update

```
Question.objects.filter(pk=1).update(
    question_text="Updated question"
)
```

Delete

```
Question.objects.filter(pk=1).delete()
```

Count

```
Question.objects.count()
```

Useful ORM Lookups

Lookup	Meaning
`exact`	Exact match
`contains`	Case-sensitive containment

`icontains`	Case-insensitive containment
`gt`	Greater than
`gte`	Greater than or equal
`lt`	Less than
`lte`	Less than or equal
`in`	Matches values in a collection
`isnull`	Checks for NULL

Example:

```
Question.objects.filter(pk__gt=5)
```

Register Models in Django Admin

File: `polls/admin.py`

```
from django.contrib import admin
from .models import Choice, Question


class ChoiceInline(admin.TabularInline):
    model = Choice
    extra = 2
```

```
@admin.register(Question)
class QuestionAdmin(admin.ModelAdmin):
    list_display = ("question_text", "pub_date")
    search_fields = ("question_text",)
    inlines = [ChoiceInline]


@admin.register(Choice)
class ChoiceAdmin(admin.ModelAdmin):
    list_display = ("choice_text", "question", "votes")
    list_filter = ("question",)
```

Create an admin account:

```
python manage.py createsuperuser
```

Open: http://127.0.0.1:8000/admin/

বাংলা ব্যাখ্যা: Model দিয়ে database table-এর structure তৈরি হয়। Migration সেই structure database-এ প্রয়োগ করে। ORM দিয়ে Python syntax ব্যবহার করে database-এর CRUD operations করা যায়।

---

4. Part 3 — Views and Templates

Function-Based Views

```
from django.shortcuts import render


def home(request):
    return render(request, "home.html")
```

`render()` combines a template with context data and returns an HTTP response.

Passing Context to a Template

```
def home(request):
    context = {
        "name": "Faruk",
        "role": "Backend Developer",
    }

    return render(request, "home.html", context)
```

Template Variables

File: `home.html`

```
<h1>Hello, {{ name }}</h1>
<p>Role: {{ role }}</p>
```

Template variables display data passed to the template.

Template Tags

If

```
{% if questions %}
    <p>Questions are available.</p>
{% else %}
    <p>No questions found.</p>
{% endif %}
```

For

```
{% for question in questions %}
    <p>{{ question.question_text }}</p>
{% empty %}
    <p>No questions available.</p>
{% endfor %}
```

Dynamic URLs

Named URLs reduce hard-coded URL strings.

In `polls/urls.py`:

```
from django.urls import path
from . import views

app_name = "polls"

urlpatterns = [
    path("", views.home, name="home"),
    path("<int:pk>/", views.detail, name="detail"),
]
```

In `polls/views.py`:

```
from django.shortcuts import get_object_or_404, render
from .models import Question


def detail(request, pk):
    question = get_object_or_404(Question, pk=pk)
```

```
    return render(
        request,
        "polls/detail.html",
        {"question": question},
    )
```

The URL parameter name `pk` must match the view argument `pk`.

Static Templates and Template Directories

Recommended structure:

```
polls/
└── templates/
    └── polls/
        ├── index.html
        ├── detail.html
        └── results.html
```

Using an application-specific directory helps prevent template-name collisions.

বাংলা ব্যাখ্যা: View request process করে এবং প্রয়োজন অনুযায়ী model থেকে data নিয়ে template-এ পাঠায়। Template HTML দিয়ে সেই data user-এর কাছে প্রদর্শন করে।

---

5. Part 4 — Forms, Voting and Generic Views

Django Forms

Forms validate submitted input and provide error messages.

File: `polls/forms.py`

```
from django import forms


class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(
        widget=forms.Textarea
    )
```

Handling a Form

File: `polls/views.py`

```
from django.shortcuts import render
from .forms import ContactForm


def contact(request):
    if request.method == "POST":
        form = ContactForm(request.POST)

        if form.is_valid():
            name = form.cleaned_data["name"]
            email = form.cleaned_data["email"]

            # Process the validated data here.

            return render(
                request,
                "contact_success.html",
                {"name": name, "email": email},
            )
    else:
        form = ContactForm()

    return render(
        request,
        "contact.html",
        {"form": form},
    )
```

Template: `contact.html`

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
</form>
```

Always validate submitted data on the server. Client-side validation alone is not sufficient.

ModelForm

A ModelForm creates a form based on a Django model.

```
from django import forms
from .models import Question
```

```
class QuestionForm(forms.ModelForm):
    class Meta:
        model = Question
        fields = ["question_text"]
```

Use an explicit list of fields rather than exposing every model field to users.

Class-Based Views

```
from django.http import HttpResponse
from django.views import View


class HomeView(View):
    def get(self, request):
        return HttpResponse("Home Page")

    def post(self, request):
        return HttpResponse("Form submitted")
```

URL configuration:

```
path("", HomeView.as_view(), name="home")
```

Generic Class-Based Views

ListView

```
from django.views.generic import ListView
from .models import Question


class QuestionListView(ListView):
    model = Question
    template_name = "polls/index.html"
    context_object_name = "questions"
    ordering = ["-pub_date"]
    paginate_by = 10
```

DetailView

```
from django.views.generic import DetailView
from .models import Question
```

```
class QuestionDetailView(DetailView):
    model = Question
    template_name = "polls/detail.html"
    context_object_name = "question"
```

CreateView

```
from django.urls import reverse_lazy
from django.views.generic import CreateView
from .models import Question


class QuestionCreateView(CreateView):
    model = Question
    fields = ["question_text"]
    template_name = "polls/question_form.html"
    success_url = reverse_lazy("polls:home")
```

UpdateView

```
from django.urls import reverse_lazy
from django.views.generic import UpdateView
from .models import Question


class QuestionUpdateView(UpdateView):
    model = Question
    fields = ["question_text"]
    template_name = "polls/question_form.html"
    success_url = reverse_lazy("polls:home")
```

DeleteView

```
from django.urls import reverse_lazy
from django.views.generic import DeleteView
from .models import Question


class QuestionDeleteView(DeleteView):
    model = Question
    template_name = "polls/question_confirm_delete.html"
    success_url = reverse_lazy("polls:home")
```

Deletion should be performed through POST after confirmation, not through a GET request.

Voting with F() Expressions

```
from django.db.models import F

Choice.objects.filter(
    pk=choice.pk,
    question=question,
).update(
    votes=F("votes") + 1
)
```

`F()` allows the database to increment the current value and helps avoid lost updates. However, `F()` does not prevent duplicate voting. If only one vote per user is allowed, store voting records and enforce uniqueness with a database constraint.

বাংলা ব্যাখ্যা: Forms user input validate করে। Generic views দিয়ে কম code লিখে list, detail, create, update এবং delete functionality তৈরি করা যায়।

---

6. Part 5 — Automated Testing

Django provides a test framework built on Python's `unittest`.

File: `polls/tests.py`

```
from django.test import TestCase
from django.urls import reverse
from django.utils import timezone

from .models import Choice, Question


class PollsTests(TestCase):
    def setUp(self):
        self.question = Question.objects.create(
            question_text="What is your favorite language?",
            pub_date=timezone.now(),
        )

        self.choice = Choice.objects.create(
            question=self.question,
            choice_text="Python",
        )

    def test_index_returns_200(self):
```

```
        response = self.client.get(
            reverse("polls:home")
        )

        self.assertEqual(response.status_code, 200)

    def test_index_contains_question(self):
        response = self.client.get(
            reverse("polls:home")
        )

        self.assertContains(
            response,
            "What is your favorite language?",
        )

    def test_missing_question_returns_404(self):
        response = self.client.get(
            reverse("polls:detail", args=[999999])
        )

        self.assertEqual(response.status_code, 404)
```

These tests assume that the `polls:home` URL displays questions and that the detail view returns 404 for a missing question.

What Should Be Tested?

- HTTP status codes
- Template rendering
- Model behavior
- Form validation
- Successful form submission
- Invalid input
- Authentication and permissions
- Voting behavior
- Duplicate-vote prevention, if required
- Database constraints

Run tests:

```
python manage.py test
```

Run tests for one application:

```
python manage.py test polls
```

বাংলা ব্যাখ্যা: Automated tests application-এর expected behavior যাচাই করে এবং code পরিবর্তনের পরে পুরোনো functionality ভেঙে গেছে কি না তা ধরতে সাহায্য করে।

---

7. Part 6 — Static Files

Static files include CSS, JavaScript, and images that are served separately from dynamic application responses.

Recommended Structure

```
polls/
├── static/
│   └── polls/
│       ├── css/
│       │   └── style.css
│       ├── js/
│       │   └── app.js
│       └── images/
│           └── logo.png
└── templates/
    └── polls/
        └── index.html
```

Configure Static Files

In `settings.py`:

```
STATIC_URL = "static/"
```

For production collection:

```
STATIC_ROOT = BASE_DIR / "staticfiles"
```

Load Static Files in a Template

```
{% load static %}

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
```

```
    <title>Django Application</title>
    <link rel="stylesheet" href="{% static 'polls/css/style.css' %}">
</head>
<body>
    <h1>Django Application</h1>
</body>
</html>
```

File: `polls/static/polls/css/style.css`

```
body {
    font-family: Arial, sans-serif;
    margin: 30px;
}

h1 {
    color: #2457a7;
}
```

Collect Static Files

```
python manage.py collectstatic
```

In production, configure a suitable static-file server or storage backend. Do not rely on Django's development static-file serving in production.

বাংলা ব্যাখ্যা: CSS, JavaScript এবং image-এর মতো file-কে static files বলা হয়। Production-এ এগুলো সঠিকভাবে serve করার configuration প্রয়োজন।

---

8. Part 7 — Django Admin Customization

Django Admin allows authorized staff to manage application data.

Basic Registration

```
from django.contrib import admin
from .models import Question

admin.site.register(Question)
```

Customize List Display, Search and Filtering

```
from django.contrib import admin
from .models import Question

```

```
@admin.register(Question)
class QuestionAdmin(admin.ModelAdmin):
    list_display = ("question_text", "pub_date")
    search_fields = ("question_text",)
    list_filter = ("pub_date",)
    ordering = ("-pub_date",)
    readonly_fields = ("pub_date",)
```

Inline Models

```
from django.contrib import admin
from .models import Choice, Question


class ChoiceInline(admin.TabularInline):
    model = Choice
    extra = 2


@admin.register(Question)
class QuestionAdmin(admin.ModelAdmin):
    list_display = ("question_text", "pub_date")
    inlines = [ChoiceInline]
```

Inline editing allows related objects to be managed from a parent model's admin page.

Admin Best Practices

- Register models intentionally.
- Use `list_display` for useful summaries.
- Add appropriate search and filter fields.
- Restrict admin access to authorized staff.
- Avoid exposing sensitive fields unnecessarily.
- Use read-only fields for data that should not be edited through admin forms.

Read-only fields do not replace application-level security or database integrity controls.

বাংলা ব্যাখ্যা: Django Admin customization করলে staff users সহজে data search, filter, update এবং related records manage করতে পারে।

---

9. Part 8 — Third-party Packages and Debug Toolbar

Third-party packages extend Django with additional functionality.

Examples:

- Django REST Framework
- Django Debug Toolbar
- Django Filter
- Pillow for image processing

Install Django Debug Toolbar

```
python -m pip install django-debug-toolbar
```

Record dependencies:

```
python -m pip freeze > requirements.txt
```

For team projects, consider maintaining a deliberate dependency file or lockfile.

Step 1: Add to Installed Apps

In `settings.py`:

```
INSTALLED_APPS = [
    # Existing Django apps...
    "debug_toolbar",
]
```

Step 2: Add Middleware

Add the middleware near the beginning of `MIDDLEWARE`, following the package's installation documentation.

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "debug_toolbar.middleware.DebugToolbarMiddleware",
    # Remaining middleware...
]
```

Preserve the middleware already required by your project.

Step 3: Configure Internal IPs

For local development:

```
INTERNAL_IPS = [
    "127.0.0.1",
]
```

Step 4: Add Toolbar URLs

In `mysite/urls.py`:

```
from django.conf import settings
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("polls/", include("polls.urls")),
]

if settings.DEBUG:
    import debug_toolbar

    urlpatterns += [
        path("__debug__/", include(debug_toolbar.urls)),
    ]
```

Preserve existing URL routes when applying this example.

What Does Debug Toolbar Provide?

- SQL query inspection
- Request and response information
- Template rendering details
- Cache information
- Timing and performance diagnostics

Security warning: Use Debug Toolbar only in controlled development environments. Do not expose it publicly in production.

বাংলা ব্যাখ্যা: Debug Toolbar database queries, request processing এবং performance সমস্যা বুঝতে সাহায্য করে। এটি production-এ public access-এর জন্য চালু রাখা উচিত নয়।

---

10. First Contribution to Django

Contributing to Django or another open-source project requires a reproducible development environment, focused changes, and tests.

Official Resources

- Django documentation: https://docs.djangoproject.com/
- Contribution guide: https://docs.djangoproject.com/en/stable/internals/contributing/
- Django GitHub repository: https://github.com/django/django

Step 1: Understand the Project

Read the official contribution guide and code of conduct.

Step 2: Fork the Repository

Open the Django GitHub repository and fork it to your GitHub account.

Step 3: Clone Your Fork

Replace `YOUR_USERNAME` with your GitHub username.

```
git clone https://github.com/YOUR_USERNAME/django.git
cd django
```

Add the official repository as the upstream remote:

```
git remote add upstream https://github.com/django/django.git
```

Step 4: Create a Virtual Environment

```
python -m venv .venv
```

Activate it using the command appropriate for your operating system.

Step 5: Install Development Dependencies

Follow Django's current contributor instructions for installing development requirements. The exact commands can vary by branch and contributor setup.

Step 6: Create a Feature Branch

```
git switch -c fix-small-bug
```

Use a descriptive branch name related to the issue.

Step 7: Choose an Issue

Look for a clearly described, reproducible bug or documentation improvement appropriate for a first contribution.

Before starting, check whether the issue is already being addressed and follow the project's contribution guidelines.

Step 8: Reproduce the Problem

Write a minimal example or test that demonstrates the problem. Reproducing the issue helps ensure that the fix addresses actual behavior rather than an assumption.

Step 9: Make a Small Change

Prefer focused changes with a clear purpose.

Step 10: Run Tests

Follow the project's documented testing instructions. A bug fix should normally include a regression test when practical.

Step 11: Review Your Changes

```
git diff
git status
```

Check for unrelated changes, missing tests, formatting problems, and unintended behavior.

Step 12: Commit Your Changes

```
git add .
git commit -m "Fix a small issue"
```

Use a concise, descriptive commit message consistent with the project's conventions.

Step 13: Push Your Branch

```
git push origin fix-small-bug
```

Step 14: Open a Pull Request

Open a pull request from your fork and branch to the appropriate upstream branch.

Include:

- A description of the problem
- The cause of the issue
- The change you made
- Tests performed
- Related issue references, if applicable

Respond constructively to review feedback and update your branch as requested.

বাংলা ব্যাখ্যা: Open-source contribution-এর মূল ধাপ হলো issue বোঝা, bug reproduce করা, ছোট change করা, test চালানো এবং pull request তৈরি করা।

---

11. Important Commands and Practice Project

Essential Django Commands

Command	Purpose

`django-admin startproject mysite`	Create a project
`python manage.py startapp polls`	Create an application
`python manage.py runserver`	Start development server
`python manage.py check`	Check project configuration
`python manage.py makemigrations`	Create migrations
`python manage.py migrate`	Apply migrations
`python manage.py showmigrations`	Display migration status
`python manage.py`	Create admin account

`createsuperuser`	
`python manage.py shell`	Open Django shell
`python manage.py test`	Run automated tests
`python manage.py collectstatic`	Collect static files

Practice Project: Blog Application

Build a complete Blog application using the concepts in these notes.

Suggested Models

- Post
- Category
- Tag
- Comment

Core Features

1. List blog posts.
2. View individual posts.
3. Create posts.
4. Update posts.
5. Delete posts with confirmation.
6. Add categories and tags.
7. Implement search.
8. Add pagination.
9. Add authentication.
10. Restrict editing to authorized users.
11. Upload and display images.

12. Write automated tests.
13. Customize Django Admin.
14. Deploy using production-appropriate settings.

Suggested Learning Order

1. Build models and migrations.
2. Register models in Django Admin.
3. Implement `ListView` and `DetailView`.
4. Add create, update, and delete operations.
5. Add forms and validation.
6. Implement authentication and permissions.
7. Add static files and media handling.
8. Write automated tests.
9. Optimize database queries.
10. Prepare the application for deployment.

Final Revision Checklist

- Understand Django's MVT architecture.
- Create projects and applications.
- Configure URL routing.
- Write function-based and class-based views.
- Create models and migrations.
- Practice ORM filtering and relationships.
- Build templates and forms.
- Use generic CBVs for CRUD operations.
- Customize Django Admin.
- Configure static files.
- Install and configure development packages.
- Write automated tests.
- Understand Git and GitHub contribution workflows.
- Build and test a complete Blog application.
- Apply appropriate security and deployment practices.

Final Goal

Build a maintainable Django application, validate user input, protect application data, test important behavior, and contribute confidently to open-source projects.

Practice + Testing + Debugging = Strong Django Skills